HOCHSCHULE KONSTANZ TECHNIK, WIRTSCHAFT UND GESTALTUNG (HTWG)
Fakultät Informatik
Rechner- und Kommunikationsnetze
Prof. Dr. Dirk Staehle

Vorlesung Rechnernetze
AIN 5

Laborübung

Paketanalyse mit Wireshark – Teil 1
(Teilaufgaben 3 bis 5)

Prof. Dr. Dirk Staehle

Die Abgabe erfolgt durch Hochladen der bearbeiteten Word-Datei in Moodle.

Bearbeitung in Zweier-Teams

Team-Mitglied 1:

Team-Mitglied 2:

3 Ruhezustand und HTTP im Klartext

In diesem Abschnitt erstellen Sie zwei Mitschnitte: einen, in dem Sie selbst nichts tun, und einen mit HTTP-Verkehr im Klartext.

Hinweis: Für diesen Teil gelten die Hinweise zur Bearbeitung und zum Datenschutz aus der Einleitung von Teil 1.1.

3.1 Mitschnitt 1: Ruhezustand

Ihr Rechner sendet auch dann Pakete, wenn Sie selbst nichts tun. Schließen Sie alle Anwendungen und starten Sie dann den Mitschnitt auf der Schnittstelle, über die Ihr Rechner mit dem Netz verbunden ist. Schneiden Sie den Verkehr so lange mit, bis Sie einige (>10) Pakete sehen. Das kann etwas dauern oder sehr schnell gehen, je nachdem, was Ihr Rechner gerade im Hintergrund macht.

1. Notieren Sie bis zu fünf Protokolle (Spalte „Protocol“ in Wireshark), die Ihr Rechner nutzt, und recherchieren Sie, was diese Protokolle machen und welcher Protokollschicht sie zugeordnet werden. Mit welcher IP-Adresse haben diese Protokolle kommuniziert?

2. Welche IPv4- und ggf. IPv6-Adresse sowie welche MAC-Adresse hat Ihr Rechner laut Mitschnitt? Vergleichen Sie mit der Ausgabe von `ipconfig /all` (Windows) bzw. `ip addr` (Linux) oder `ifconfig` (macOS).

3. An welche Ziel-MAC-Adresse sendet Ihr Rechner Pakete? Vergleichen Sie die Ziel-MAC-Adresse für verschiedene Ziel-IP-Adressen. Welchem Netzknoten können Sie die Ziel-MAC-Adresse zuordnen?

3.2 Mitschnitt 2: HTTP im Klartext

Starten Sie einen neuen Mitschnitt und erzeugen Sie HTTP-Verkehr nach einer der folgenden Varianten (gerne auch mehreren). Beenden und speichern Sie den Mitschnitt anschließend. Die meisten Webseiten werden heute nur noch verschlüsselt ausgeliefert, und Browser versuchen zunehmend, automatisch auf HTTPS umzustellen. Die Varianten sind so gewählt, dass Sie trotzdem unverschlüsselten Verkehr erhalten.

Variante A: Browser. Rufen Sie im Browser die folgende Seite auf. Geben Sie `http://` ausdrücklich mit ein.

```
http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html
```

Aktuelle Browser versuchen zunächst automatisch eine verschlüsselte Verbindung aufzubauen und warnen teilweise vor unverschlüsselten Seiten. So erzwingen Sie HTTP:

- **Chrome:** Seit Version 154 warnt Chrome vor dem Aufruf öffentlicher Seiten über HTTP. Klicken Sie in der Warnung auf „Weiter zur Website“ oder deaktivieren Sie unter `chrome://settings/security` die Option „Immer sichere Verbindungen verwenden“.
- **Edge:** Suchen Sie in den Einstellungen nach „HTTPS“ und deaktivieren Sie „Automatisches HTTPS“ bzw. den automatischen Wechsel zu sichereren Verbindungen.
- **Firefox:** Firefox versucht seit Version 136 zunächst HTTPS und fällt auf HTTP zurück, wenn der Server kein HTTPS anbietet. Falls Sie den „Nur-HTTPS-Modus“ aktiviert haben (Einstellungen → Datenschutz & Sicherheit), schalten Sie ihn für die Übung aus.
- **Safari:** Wenn unter Safari → Einstellungen → Sicherheit „Vor dem Verbinden mit einer Website über HTTP warnen“ aktiviert ist, bestätigen Sie die Warnung.

Variante B: curl. Rufen Sie dieselbe Seite im Terminal bzw. in der Eingabeaufforderung ab. curl stellt nie automatisch auf HTTPS um.

```
curl http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html
```

Variante C: Eigener Webserver auf Ihrem Rechner. Starten Sie in einem beliebigen Verzeichnis einen Webserver (unter Windows ggf. `python` oder `py` statt `python3`):

```
python3 -m http.server 8000
```

Rufen Sie im Browser auf demselben Rechner `http://127.0.0.1:8000` auf. Dieser Verkehr verlässt Ihren Rechner nicht. Sie müssen in Wireshark deshalb die Loopback-Schnittstelle mitschneiden statt der WLAN-Schnittstelle: unter Windows „Adapter for loopback traffic capture“, unter macOS „lo0“, unter Linux „lo“.

Kurz erklärt: Request, Response und Verbindung. Beim Abruf einer Webseite schickt Ihr Rechner (der *Client*) eine Anfrage an den Webserver, den **HTTP-Request** (z. B. `GET /index.html`). Der Server antwortet mit der **HTTP-Response**, die die Seite enthält.

Bevor Ihr Rechner den Request senden kann, muss er zwei Dinge erledigen: Er muss die IP-Adresse des Servers kennen; falls er sie noch nicht kennt, fragt er sie per **DNS** beim Nameserver ab. Außerdem baut er eine **TCP-Verbindung** zum Server auf. Eine Verbindung ist eine

Vereinbarung zwischen zwei Anwendungen, miteinander Daten auszutauschen: Zuerst stimmen sich beide Seiten mit drei kurzen Paketen ab (Verbindungsaufbau, in Wireshark als `SYN`, `SYN, ACK` und `ACK` angezeigt), dann werden die Daten übertragen, und am Ende wird die Verbindung mit `FIN`-Paketen wieder abgebaut. Wie das im Detail funktioniert, behandeln wir später in der Vorlesung.

4. Suchen Sie mit dem Filter `http` Ihren HTTP-Request (in der Spalte „Info" steht `GET …`) und merken Sie sich die Paketnummer. Entfernen Sie den Filter wieder und betrachten Sie die etwa zehn Pakete vor dem Request: Quell- und Ziel-IP-Adresse, Protokoll, Portnummern und Inhalt (Spalte „Info"). Welche dieser Pakete könnten mit Ihrem Request zusammenhängen, und was ist dort vermutlich passiert?
   Tipp: Achten Sie auf Pakete mit der IP-Adresse des Servers und auf DNS-Pakete mit dem Namen `gaia.cs.umass.edu`. Je nach Variante finden Sie nicht alles davon: Warum?

5. Öffnen Sie mit Rechtsklick auf den Request → Folgen → TCP-Stream die gesamte Verbindung. Wireshark zeigt den Inhalt der Verbindung in einem eigenen Fenster und in der Paketliste nur noch die Pakete dieser Verbindung. Finden Sie in der Paketliste den Verbindungsaufbau, den HTTP-Request, die HTTP-Response und den Verbindungsabbau (siehe Kasten oben). Aus wie vielen Paketen besteht die Verbindung?
   Hinweis: Browser halten Verbindungen oft noch eine Weile offen. Falls Sie keinen Verbindungsabbau finden, notieren Sie das einfach.

6. Bestimmen Sie aus den Zeitstempeln in der Spalte „Time", wie lange es vom Senden des HTTP-Requests bis zum Eintreffen der HTTP-Response gedauert hat. Wählen Sie anschließend die HTTP-Response aus und klappen Sie in den Paketdetails den HTTP-Teil auf. Welche Angabe finden Sie dort, die Ihre Rechnung bestätigt? *[Diskussion]* Einige Angaben stehen in eckigen Klammern. Was bedeutet das, und sind diese Angaben im Paket tatsächlich enthalten?

7. Vergleichen Sie Ihren HTTP-Request mit dem Paket aus dem Hex-Dump in Abschnitt 2 in drei Punkten: Ziel-Port, Zeile `Host:` und Zeile `User-Agent:` im HTTP-Teil. Was verrät der User-Agent jeweils über das Programm (und ggf. das Betriebssystem), das den Request geschickt hat?

4 Filter

Display-Filter haben Sie in den bisherigen Aufgaben bereits benutzt (z. B. `icmp`, `dns` und `http`). In diesem Abschnitt vertiefen Sie den Umgang damit.

Wireshark kennt zwei Arten von Filtern. Capture-Filter legen fest, welche Pakete überhaupt aufgezeichnet werden. Display-Filter legen fest, welche der aufgezeichneten Pakete angezeigt werden. Beide haben eine unterschiedliche Syntax.

Display-Filter filtern nach Protokollen und nach dem Inhalt von Header-Feldern. Der Filter `ip.addr==192.140.168.15` zeigt alle Pakete mit dieser Quell- oder Zieladresse, `udp.port==1500` alle Pakete mit UDP-Port 1500 auf Sender- oder Empfängerseite. Bedingungen lassen sich mit `&&`, `||` und `!` verknüpfen, Werte mit `==`, `!=`, `<`, `>` oder `contains` vergleichen. Hilfreich sind die Autovervollständigung im Filterfeld (z. B. nach Eingabe von `tcp.`) und die Übersicht unter https://www.wireshark.org/docs/dfref/.

Schneiden Sie 30 bis 60 Sekunden lang mit, während Sie einige verschiedene Webseiten aufrufen (z. B. eine Suchmaschine, ein Video-Portal, eine Nachrichtenseite).

1. Formulieren Sie für die folgenden Aufgaben jeweils einen Display-Filter und notieren Sie, wie viele Pakete er anzeigt. Tipp: Klicken Sie in den Paketdetails auf das gesuchte Feld. Den Feldnamen sehen Sie in der Statusleiste, und mit Rechtsklick → Als Filter anwenden übernehmen Sie ihn in das Filterfeld.

Aufgabe	**Filter**	**Anzahl Pakete**
Alle Pakete, die an Ihren Rechner gerichtet sind (IPv4 und IPv6)		
Alle DNS-Anfragen, aber keine DNS-Antworten (Tipp: Feld `dns.flags.response`)		
Alle DNS-Anfragen nach einem Namen, der den Namen einer von Ihnen besuchten Seite enthält (Tipp: Operator `contains`)		

Aufgabe	Filter	Anzahl Pakete
Alle Pakete, die größer als 1000 Byte sind und an Ihren Rechner gehen		

2. Öffnen Sie Ihren Mitschnitt mit HTTP-Verkehr aus Abschnitt 3. Vergleichen Sie die Ergebnisse der Filter `http` und `tcp.port==80`. Welche Pakete zeigt nur der zweite Filter an, und warum? *[Diskussion]* Was beobachten Sie, wenn Sie in Abschnitt 3 Variante C (Port 8000) genutzt haben?

5 Verschlüsselter HTTP-Verkehr (HTTPS)

HTTPS-Verkehr läuft entweder über TCP (HTTP/1.1 und HTTP/2, verschlüsselt mit TLS) oder über UDP (HTTP/3 mit QUIC), beides standardmäßig auf Port 443. Starten Sie einen neuen Mitschnitt und rufen Sie im Browser nacheinander `https://www.htwg-konstanz.de` und `https://www.google.com` auf; laden Sie jede Seite zweimal.

5.1 Analyse von verschlüsseltem Verkehr

1. Wie viele Pakete zeigen die Filter `tcp.port==443`, `udp.port==443`, `tls` und `quic` jeweils an? Welche der beiden Seiten nutzt QUIC? Welcher Filter erfasst den gesamten HTTPS-Verkehr?

2. *[Diskussion]* Es gibt einen Filter `http`, aber keinen Filter `https`. Warum?

3. Suchen Sie mit dem Filter `tls.handshake.type==1` die Client-Hello-Pakete, also jeweils das erste Paket des TLS-Handshakes. Wählen Sie das Client Hello an www.htwg-konstanz.de aus. Den Servernamen finden Sie in den Paketdetails unter Transport Layer Security → Handshake Protocol → Extension: server_name (SNI). Notieren Sie Ziel-IP-Adresse, Ziel-Port und Servernamen.

4. Füllen Sie die folgende Tabelle für einen Beobachter im Netz aus:

Information	**sichtbar (ja / nein / teilweise)**	**woran erkennbar?**
IP-Adresse des Servers		
Ports und Transportprotokoll		
Domainname (z. B. www.beispiel.de)		
Vollständige URL inkl. Pfad		
HTTP-Header (z. B. User-Agent, Cookies)		
Inhalt der Seite		
Übertragene Datenmenge		
Zeitpunkte und Dauer des Abrufs		

5.2 Entschlüsselung des eigenen HTTPS-Verkehrs

Firefox und Chrome-basierte Browser können die TLS-Sitzungsschlüssel in eine Datei schreiben, wenn die Umgebungsvariable `SSLKEYLOGFILE` gesetzt ist. Wireshark kann damit den Verkehr entschlüsseln. Safari unterstützt das nicht; nutzen Sie für diesen Abschnitt Firefox, Chrome oder Edge.

ACHTUNG: Sobald die Entschlüsselung eingerichtet ist, zeigt Wireshark alles im Klartext an, was Ihr Browser sendet und empfängt, also auch Passwörter, Cookies und Login-Daten, und zwar auch von Tabs, die nebenbei geöffnet sind. Melden Sie sich während der Übung nirgends an und nutzen Sie ein privates Fenster. Mit der Datei `keys.log` und einem Mitschnitt kann jeder Ihren Verkehr entschlüsseln. Geben Sie beides nicht weiter und laden Sie es nicht hoch. **Entfernen Sie nach der Übung die Umgebungsvariable und löschen Sie die Datei.**

- **Windows:** Suchen Sie im Startmenü nach „Umgebungsvariablen für dieses Konto bearbeiten“ und legen Sie unter Benutzervariablen eine neue Variable `SSLKEYLOGFILE` mit einem Pfad wie `C:\Users\<name>\keys.log` an.
- **macOS/Linux:** Setzen Sie im Terminal `export SSLKEYLOGFILE=~/keys.log` und starten Sie den Browser aus demselben Terminal, z. B. mit `firefox &` (Linux) oder `/Applications/Firefox.app/Contents/MacOS/firefox &` (macOS).
- **Browser neu starten:** Beenden Sie den Browser vollständig und starten Sie ihn neu. Chrome und Edge laufen unter Windows oft im Hintergrund weiter; beenden Sie sie dann im Task-Manager. Rufen Sie eine Seite auf und prüfen Sie, dass die Datei `keys.log` entstanden ist und wächst.
- **Wireshark:** Tragen Sie unter Bearbeiten → Einstellungen → Protocols → TLS im Feld „(Pre)-Master-Secret log filename“ dieselbe Datei ein (unter macOS: Wireshark → Einstellungen).

5. Starten Sie einen neuen Mitschnitt und rufen Sie die beiden Seiten im privaten Fenster erneut auf. Welche Protokolle zeigt Wireshark jetzt oberhalb von TLS bzw. QUIC an? Finden Sie einen Request, der den vollständigen Pfad einer abgerufenen Ressource enthält? Fügen Sie einen Screenshot ein.

6. *[Diskussion]* Warum funktioniert die Entschlüsselung nur auf Ihrem eigenen Rechner und nicht für einen Beobachter im Netz? Was würde es bedeuten, wenn jemand Zugriff auf Ihre Datei keys.log hätte? Löschen Sie die Umgebungsvariable und die Datei nach der Übung wieder.